EVALUACIÓN SOLEMNE: SISTEMA IoT CON MICROSERVICIOS EN RASPBERRY PI Y PLATAFORMA EN LA NUBE

OBJETIVO GENERAL

Diseñar, implementar y desplegar un sistema IoT que capture datos del mundo físico con sensores conectados a una **Raspberry Pi 3**, los procese mediante **microservicios en contenedores** y los almacene en una **base de datos en la nube**, ofreciendo una plataforma web para consultarlos, y realizar una **evaluación realista del uso y del volumen de datos** que genera el sistema, contrastando lo estimado con lo medido y con los límites de los servicios utilizados.

DESCRIPCIÓN DE LA ACTIVIDAD

Cada grupo desarrollará un sistema IoT que resuelva una problemática real o simulada. Una Raspberry Pi 3 con al menos dos sensores capturará datos en terreno; sobre ella se ejecutarán varios microservicios en contenedores, encargados de leer los sensores, procesar las lecturas y enviarlas a la nube; los datos se almacenarán en Cloud Firestore; y una plataforma web desplegada en Vercel permitirá a los usuarios visualizarlos en tiempo real, consultar históricos y configurar los dispositivos.

Además del desarrollo, cada grupo deberá **evaluar con datos reales cuánto usa su sistema**: cuántas lecturas, escrituras, consultas y almacenamiento genera, cuánto tiempo puede operar dentro de la capa gratuita, en qué punto la superaría al crecer y qué decisiones de diseño reducen ese consumo.

Se deberán formar **grupos de máximo 5 estudiantes**. Cada grupo debe contar con **al menos una Raspberry Pi 3**, que puede solicitarse en préstamo a la universidad. Cada grupo debe elegir uno de los proyectos propuestos o proponer uno equivalente, validado por el docente.

STACK TECNOLÓGICO PROPUESTO

El siguiente stack permite desarrollar el proyecto completo **sin costo y sin tarjeta de crédito**, usando las capas gratuitas de **Vercel** y **Firebase** como plataformas (PaaS). Se pueden usar alternativas equivalentes siempre que cumplan la regla de costo cero y sean validadas por el docente.

Componente	Tecnología	Costo / condición
Dispositivo IoT	Raspberry Pi 3 con Raspberry Pi OS (64 bits) y dos o más sensores	Préstamo de la universidad. Sensores de los kits disponibles en laboratorio.
Microservicios	Docker y Docker Compose en la Raspberry Pi; mensajería interna con MQTT (Mosquitto) o REST	Gratis.
Plataforma web y API	Vercel, plan Hobby (por ejemplo, Next.js con API routes)	Gratis, sin tarjeta. Uso académico (no comercial).

Componente	Tecnología	Costo / condición
Base de datos y autenticación	Firebase, plan Spark: Cloud Firestore + Firebase Authentication	Gratis, sin tarjeta. Firestore: 50.000 lecturas, 20.000 escrituras y 20.000 eliminaciones diarias; 1 GiB almacenado.
Código e imágenes	GitHub y GitHub Container Registry	Gratis.
Alertas (opcional)	Bots de Telegram o Discord (webhooks), correo (Resend), Open-Meteo (clima)	Capas gratuitas.

Regla de costo cero

- **Ningún servicio del proyecto debe requerir ingresar una tarjeta de crédito.** Si un servicio la solicita, se debe buscar una alternativa.
- Para el hardware se deben priorizar las Raspberry Pi y los sensores disponibles en la universidad.
- En **Firebase no se deben usar Cloud Functions ni Cloud Storage**, ya que ambos exigen el plan Blaze (con tarjeta). La lógica de servidor se implementa en las API routes de Vercel.
- No usar autenticación por SMS (tiene costo). Usar correo/contraseña o proveedores como Google.
- Si el sistema supera la capa gratuita durante las pruebas, Firebase deja de aceptar operaciones hasta el día siguiente; no se generan cobros. Evitarlo es parte del diseño y de la evaluación de uso.

REQUISITOS MÍNIMOS

1. Sistema IoT en la Raspberry Pi

- Al menos una Raspberry Pi 3 con **dos o más sensores de tipos distintos** que generen datos reales, por ejemplo: temperatura y humedad (DHT22 o BME280), distancia por ultrasonido (HC-SR04), presencia o proximidad infrarroja (PIR HC-SR501 o módulo de obstáculos IR), luz (BH1750) o presión (BMP280).
- Las lecturas de los sensores deben combinarse para generar información útil (por ejemplo, ocupación de una sala a partir de presencia y temperatura), no solo mostrarse por separado.
- El dispositivo recibe **configuración desde la plataforma web** (por ejemplo, intervalo de muestreo, umbrales de alerta o activación de cada sensor) y la aplica sin reiniciarse.
- El dispositivo **solo inicia conexiones salientes** por HTTPS. No se permite abrir puertos entrantes, exponer la Raspberry Pi a Internet ni usar túneles desde la red institucional.
- **Resiliencia:** si se pierde la conexión, las lecturas se guardan localmente (por ejemplo, en SQLite) y se reenvían al reconectarse, sin duplicados.
- **Cuidado con los voltajes:** los pines GPIO trabajan a 3,3 V. Sensores como el HC-SR04 entregan 5 V en su salida y requieren un divisor de voltaje. Los sensores analógicos necesitan un conversor (por ejemplo, ADS1115), ya que la Raspberry Pi no tiene entradas analógicas.

2. Microservicios en la Raspberry Pi

El software del dispositivo debe estar dividido en **al menos tres microservicios**, cada uno en su propio contenedor, con una responsabilidad única y justificada, orquestados con **Docker Compose**. Los microservicios se comunican entre sí mediante mensajes (MQTT) o **APIs REST** internas, nunca leyendo directamente los datos de otro servicio. Una arquitectura de referencia es:

Microservicio	Responsabilidad
Adquisición (uno por sensor o por tipo de sensor)	Lee el sensor a través del GPIO según el intervalo configurado y publica cada lectura en un tópico interno (por ejemplo, sensores/temperatura).
Broker de mensajería (Mosquitto)	Desacopla a los productores y consumidores de lecturas dentro del dispositivo.
Procesamiento	Valida y filtra las lecturas, las combina y las agrega (por ejemplo, promedios por minuto) y detecta eventos o umbrales superados.
Sincronización	Envía lotes de datos a la API en la nube, mantiene el buffer local cuando no hay conexión y obtiene la configuración vigente del dispositivo.
Salud (opcional)	Expone un endpoint local con el estado de cada microservicio, uso de CPU, memoria y temperatura de la placa.

- Cada contenedor debe tener política de reinicio automático y un healthcheck, y el sistema completo debe iniciarse solo al encender la Raspberry Pi.
- Los secretos (por ejemplo, el token del dispositivo) se configuran con variables de entorno, nunca en el código ni en las imágenes.
- Consideraciones de la Raspberry Pi 3: tiene 1 GB de RAM y arquitectura ARM, por lo que se deben usar imágenes livianas compatibles con ARM (por ejemplo, python:3-slim o node:alpine) y configurar archivo de swap. Para acceder al GPIO desde un contenedor se debe mapear el dispositivo correspondiente (por ejemplo, /dev/gpiomem) en lugar de ejecutar el contenedor en modo privilegiado.

3. Almacenamiento de datos en la nube

- Las lecturas captadas por los sensores se almacenan en **Cloud Firestore**.
- El dispositivo envía los datos a una **API propia desplegada en Vercel**, que los valida y los escribe en Firestore. La Raspberry Pi **nunca** debe contener credenciales de administrador de Firebase (cuenta de servicio); estas solo existen como variables de entorno en Vercel.
- Cada dispositivo se autentica ante la API con un **token propio**, generado desde la plataforma web y revocable.
- El **modelo de datos** (colecciones, documentos, campos e índices) debe estar documentado y justificado según las consultas que realiza la plataforma y el volumen de datos esperado (ver requisito 5).

4. Plataforma web y API

- Desplegada en Vercel, con URL pública HTTPS y despliegue automático desde GitHub.
- Autenticación con Firebase Authentication y **al menos dos roles** (por ejemplo, administrador y usuario de consulta).
- Soporte para **varias organizaciones o sitios** con sus datos aislados entre sí mediante Security Rules de **Firestore**.
- Dashboard con datos en tiempo real, consulta de históricos por rango de fechas, alertas por umbrales y gestión de dispositivos (alta, token, configuración y estado de conexión).
- Arquitectura en capas (presentación, lógica de negocio y acceso a datos) y API REST documentada con OpenAPI/Swagger o con una tabla de endpoints con ejemplos en el README.

5. Evaluación de uso y capacidad

Cada grupo debe realizar una evaluación realista del consumo de su sistema, basada en sus propias decisiones de diseño y validada con mediciones reales. La evaluación debe incluir:

1. **Escenario de uso:** cantidad de dispositivos y sensores, frecuencia de muestreo, frecuencia de envío a la nube, tamaño aproximado de cada documento y cantidad de usuarios consultando el dashboard.
2. **Estimación de consumo diario y mensual:** escrituras, lecturas (incluyendo las que generan el dashboard en tiempo real y las consultas de históricos), almacenamiento acumulado e invocaciones a la API en Vercel.
3. **Medición real:** operar el sistema de forma continua durante al menos 7 días y comparar lo estimado con lo registrado en las consolas de uso de Firebase y Vercel, explicando las diferencias.
4. **Límites:** porcentaje de la capa gratuita que consume el sistema, por cuánto tiempo puede operar el almacenamiento antes de superar el límite y con cuántos dispositivos se superaría cada límite diario.
5. **Optimización:** al menos dos decisiones de diseño que reducen el consumo (por ejemplo, agregación en el dispositivo, envío por lotes, un documento por minuto con varias lecturas, política de retención o resúmenes diarios), con el consumo antes y después de aplicarlas.
6. **Proyección de crecimiento:** consumo y costo mensual estimado si el sistema creciera a 10, 100 y 1.000 dispositivos fuera de la capa gratuita, usando los precios oficiales vigentes, e identificación del primer recurso que se volvería un cuello de botella.
7. **Evaluación de costos de migración:** estimación del costo mensual que representaría migrar el sistema a una plataforma PaaS de pago (por ejemplo, Vercel Pro con Firebase en plan Blaze, Google App Engine o Azure App Service) y a una infraestructura IaaS (por ejemplo, máquinas virtuales en AWS EC2, Azure Virtual Machines o Google Compute Engine con una base de datos administrada por el grupo), para el escenario actual y para 100 dispositivos. Se deben comparar los costos de cada alternativa, el esfuerzo de administración y las responsabilidades que asumiría el grupo en cada caso (actualizaciones, respaldos, seguridad y escalamiento), usando las calculadoras de precios oficiales de cada proveedor.

El Anexo A incluye una plantilla con un ejemplo de cálculo. Los límites de las capas gratuitas deben consultarse en la documentación oficial vigente de cada servicio y citarse en el documento.

6. Integraciones (opcional)

Opcionalmente, el sistema puede integrarse con servicios externos, por ejemplo: alertas por Telegram o Discord, envío de correos o datos de clima de Open-Meteo (gratuito y sin clave de API) para enriquecer el análisis.

7. Despliegue y evidencia

- Plataforma accesible públicamente el día de la evaluación, con la Raspberry Pi presente y enviando datos en vivo.
- Se recomienda conectar la Raspberry Pi a la red de un celular (hotspot) durante la presentación, ya que la red institucional puede bloquear algunos servicios.
- Video de respaldo de la demostración (3 a 5 minutos), por si falla la conectividad durante la presentación.

PROPUESTAS DE PROYECTOS (REFERENCIALES)

Los grupos pueden elegir una de las siguientes ideas o proponer una alternativa equivalente validada por el docente.

Proyecto	Descripción	Sensores sugeridos	Microservicios sugeridos
Smart Campus: monitoreo ambiental de salas	Las Raspberry Pi miden temperatura, humedad y presencia en las salas; el panel muestra dashboards en tiempo real, históricos y un índice de confort y uso por sala, y alerta al superar umbrales configurables.	DHT22 o BME280 + sensor PIR.	Adquisición ambiental, adquisición de presencia, cálculo de índice de uso, sincronización.
Control de aforo de espacios	En la puerta de una biblioteca, laboratorio o local, un sensor IR y uno ultrasónico detectan el paso de personas y su dirección para calcular el aforo en tiempo real, alertar cuando se supera la capacidad y generar reportes de horas punta.	HC-SR04 + módulo IR de obstáculos (o dos HC-SR04).	Adquisición por sensor, detección de dirección y conteo, sincronización.
Estacionamiento inteligente	Sensores ultrasónicos detectan si cada puesto está ocupado y un sensor IR cuenta los vehículos que ingresan. Los usuarios ven los puestos libres en una página pública y el administrador obtiene estadísticas de rotación.	HC-SR04 (uno por puesto simulado) + sensor IR o PIR.	Adquisición por puesto, estado de ocupación, conteo de ingresos, sincronización.
Monitoreo de nivel de estanques o contenedores	Un sensor ultrasónico mide el nivel de llenado de contenedores de basura o estanques de agua y un sensor de temperatura detecta condiciones anormales. El sistema avisa cuándo vaciar o rellenar y proyecta cuándo se llenará cada uno.	HC-SR04 + DHT22.	Adquisición de nivel, adquisición de temperatura, proyección de llenado, sincronización.

Proyecto	Descripción	Sensores sugeridos	Microservicios sugeridos
Monitoreo de salas de servidores	Se miden la temperatura y humedad de la sala o rack y se detecta presencia o la apertura de la puerta del rack. El sistema alerta por sobrecalentamiento o accesos fuera de horario y registra cada evento.	DHT22 o BME280 + PIR o módulo IR de proximidad.	Adquisición ambiental, detección de accesos, motor de alertas, sincronización.
Monitoreo de invernaderos y huertos	Se miden la temperatura y humedad ambiental, la luz y, opcionalmente, la humedad del suelo, y se cruzan con el pronóstico del tiempo (Open-Meteo) para recomendar cuándo regar o ventilar.	DHT22 + BH1750 (luz); opcional: humedad de suelo con ADS1115.	Adquisición por sensor, motor de recomendaciones, sincronización.

ENTREGABLES

- **Repositorio de código** (GitHub) con commits de todos los integrantes, que incluya los microservicios con su archivo docker-compose.yml y la plataforma web.
- **URL pública** de la plataforma y **cuentas de prueba** con distintos roles.
- **Documento técnico** con: descripción del sistema, esquema de conexión del hardware, arquitectura de microservicios, modelo de datos, documentación de la API, medidas de seguridad e instrucciones para desplegar desde cero.
- **Informe de evaluación de uso y capacidad** (requisito 5), con las capturas de las consolas de uso de Firebase y Vercel del periodo de medición.
- **Diagrama de arquitectura** que muestre los sensores, los microservicios y su comunicación, la API, la base de datos y la plataforma web.
- **Video de respaldo** de la demostración y **presentación** usada en la defensa.

REQUISITOS EXCLUYENTES

Si no se cumple cualquiera de los siguientes requisitos, **la nota máxima del proyecto es 4,0**:

- La plataforma web funciona públicamente el día de la evaluación.
- La Raspberry Pi envía datos reales de al menos dos sensores a Firestore.
- Al menos tres microservicios se ejecutan en contenedores en la Raspberry Pi.
- El repositorio tiene commits de todos los integrantes.

RÚBRICA DE EVALUACIÓN

La evaluación tiene un total de **100 puntos**: desarrollo técnico (70 puntos) y presentación oral (30 puntos). Cada criterio se evalúa en cuatro niveles: **Logrado** (100% del puntaje), **Suficiente** (70%), **Insuficiente** (40%) y **No presenta** (0%).

Criterio	Logrado	Suficiente	Insuficiente	No presenta
DESARROLLO TÉCNICO (70 puntos)				
1. Sistema IoT y sensores (15 pts)	**15 pts.** Dos o más sensores de tipos distintos, bien conectados (voltajes correctos), con datos reales combinados en información útil. Aplica configuración remota sin reiniciarse y guarda y reenvía lecturas ante desconexiones, sin duplicados.	**10,5 pts.** Dos sensores funcionando, pero sin configuración remota o sin resiliencia ante desconexiones.	**6 pts.** Un solo sensor funcionando, datos simulados o lecturas inestables.	**0 pts.** No hay sistema IoT funcionando.
2. Microservicios en la Raspberry Pi (15 pts)	**15 pts.** Tres o más microservicios con responsabilidad única y justificada, comunicados por MQTT o REST, orquestados con Docker Compose, con healthchecks, reinicio automático e inicio al encender la placa. Imágenes ARM livianas y secretos en variables de entorno.	**10,5 pts.** Tres microservicios funcionando, pero con responsabilidades poco claras, sin healthchecks o con inicio manual.	**6 pts.** Menos de tres contenedores, o un "monolito partido" cuyos servicios comparten datos o código sin una interfaz definida.	**0 pts.** No hay microservicios.
3. Almacenamiento en la nube y modelo de datos (10 pts)	**10 pts.** Lecturas almacenadas en Firestore a través de la API, con validación. Modelo de datos documentado y justificado según las consultas y el volumen esperado, con índices definidos.	**7 pts.** Almacenamiento funcional, pero con un modelo de datos poco justificado o ineficiente.	**4 pts.** Almacenamiento parcial o escritura directa desde el dispositivo con credenciales de administrador.	**0 pts.** No se almacenan datos en la nube.
4. Plataforma web y API (10 pts)	**10 pts.** Desplegada en Vercel con despliegue automático. Autenticación con dos roles, organizaciones aisladas, dashboard en tiempo real, históricos, alertas y gestión de dispositivos. Arquitectura en capas y API documentada.	**7 pts.** Plataforma funcional a la que le falta una de las funcionalidades o con documentación incompleta de la API.	**4 pts.** Plataforma básica (solo muestra datos) o con errores importantes.	**0 pts.** No hay plataforma web.
5. Evaluación de uso y capacidad (15 pts)	**15 pts.** Incluye los siete elementos del requisito 5, incluida la evaluación de costos de migración a PaaS e IaaS, con cálculos correctos y fuentes oficiales citadas. La medición de 7 días se contrasta con la estimación y las diferencias se explican. Las optimizaciones muestran su efecto con datos.	**10,5 pts.** Estimación y medición completas, pero con optimizaciones o proyección superficiales.	**6 pts.** Solo estimaciones teóricas, sin medición real, o con errores de cálculo importantes.	**0 pts.** No presenta evaluación.
6. Seguridad y documentación (5 pts)	**5 pts.** Token propio y revocable por dispositivo, Security Rules por organización y rol, sin secretos en el repositorio ni en su historial. Documento técnico y diagrama completos; el README permite desplegar desde cero.	**3,5 pts.** Medidas de seguridad y documentación presentes con debilidades menores.	**2 pts.** Reglas abiertas (modo de prueba), secretos expuestos o documentación mínima.	**0 pts.** Sin medidas de seguridad ni documentación.

Criterio	Logrado	Suficiente	Insuficiente	No presenta
PRESENTACIÓN ORAL (30 puntos)				
7. Demostración en vivo (10 pts)	**10 pts.** Con la Raspberry Pi presente, muestra: datos llegando en tiempo real, un cambio de configuración aplicado por el dispositivo, la recuperación tras desconectarlo de la red, el estado de los microservicios y un usuario que no puede ver los datos de otra organización.	**7 pts.** Demostración completa con algún paso omitido o una falla menor.	**4 pts.** Demostración parcial o solo con el video de respaldo.	**0 pts.** No hay demostración.
8. Defensa técnica (10 pts)	**10 pts.** Explica con precisión la arquitectura, por qué dividió los microservicios de esa forma, el modelo de datos y los resultados de la evaluación de uso, y responde cómo se comportaría el sistema al crecer.	**7 pts.** Respuestas correctas, pero con poca profundidad.	**4 pts.** Respuestas vagas o con errores conceptuales.	**0 pts.** No logra explicar el sistema.
9. Pregunta individual (puntaje individual) (10 pts)	**10 pts.** El estudiante explica correctamente una parte del sistema desarrollada por otro integrante.	**7 pts.** Explica con imprecisiones.	**4 pts.** Explicación superficial o confusa.	**0 pts.** No responde.

Coevaluación

Cada integrante evaluará el aporte de sus compañeros. El resultado se aplica como un factor entre **0,8 y 1,0** sobre la nota grupal para obtener la nota individual (el criterio 9 ya es individual).

CONSIDERACIONES FINALES

El foco de esta evaluación es integrar hardware, microservicios y servicios en la nube en un sistema que funcione de verdad, y que el estudiante sea capaz de dimensionarlo: entender cuántos datos genera, cuánto cuesta almacenarlos y consultarlos, y cómo sus decisiones de diseño afectan ese consumo. Más que la cantidad de funcionalidades, se valorará que cada decisión técnica esté fundamentada con datos.

Fecha de entrega final: miércoles 28 de octubre de 2026. Cada grupo deberá subir a Canvas **un archivo comprimido con el informe y la presentación**, incluyendo en el informe el enlace al repositorio y la URL pública de la plataforma. **El entregable se cerrará el miércoles 28 de octubre de 2026 a las 23:59 h** y no se recibirán entregas posteriores. Las Raspberry Pi y los componentes facilitados por la universidad deben devolverse en buen estado al término del proyecto.

ANEXO A: PLANTILLA DE EVALUACIÓN DE USO (CON EJEMPLO)

El siguiente ejemplo considera **una Raspberry Pi con dos sensores que se leen cada 10 segundos** y compara dos diseños. Los tamaños de documento son aproximados; cada grupo debe calcular los suyos y considerar que los índices de Firestore también ocupan espacio.

Variable	Diseño A: un documento por lectura	Diseño B: un documento por minuto (promedios de ambos sensores)
Lecturas por día	2 sensores × 8.640 = 17.280	2 sensores × 8.640 = 17.280 (agregadas en el dispositivo)
Escrituras en Firestore por día	17.280 (86% del límite diario de 20.000)	1.440 (7% del límite)
Lecturas al cargar el histórico de 24 h en el dashboard	17.280 por cada carga: 3 cargas superan el límite diario de 50.000	1.440 por carga: unas 34 cargas diarias caben en el límite
Tamaño aproximado por documento	0,25 KB	0,3 KB
Almacenamiento por día	≈ 4,3 MB	≈ 0,4 MB
Almacenamiento por año	≈ 1,6 GB (supera el límite de 1 GiB en unos 8 meses)	≈ 0,16 GB
Dispositivos soportados dentro del límite de escrituras	1	13

Conclusión del ejemplo: con el diseño A, **un solo dispositivo** casi agota la capa gratuita y el dashboard deja de funcionar a las pocas consultas; agregar las lecturas en la Raspberry Pi antes de enviarlas reduce el consumo en más de 10 veces. Cada grupo debe hacer este análisis con su propio sistema, completar la tabla con los valores medidos en los 7 días de operación y agregar la proyección a 10, 100 y 1.000 dispositivos.